

HTML Interview Questions & Answers

20 questions spanning basic to advanced HTML concepts

Basics

1. What is HTML, and what is its role in web development?

HTML (HyperText Markup Language) is the standard markup language used to structure content on the web. It defines the elements of a page — headings, paragraphs, links, images, forms, and so on — using tags, which browsers then parse and render visually. It works alongside CSS (styling) and JavaScript (behavior) to form the three core building blocks of the web.

2. What is the difference between an HTML element and an HTML tag?

A tag is the markup itself, e.g. `<p>` or `</p>`, marking the start or end of a piece of content. An element is the complete unit made up of the opening tag, its content, and the closing tag (e.g. `<p>Hello</p>`), plus any attributes on the opening tag.

3. What is the difference between block-level and inline elements?

Block-level elements (e.g. `<div>`, `<p>`, `<h1>`) start on a new line and take up the full available width by default, allowing width/height/margin to be set on all sides. Inline elements (e.g. `<span>`, `<a>`, `<strong>`) flow within the surrounding text, only take up as much width as their content, and don't respect top/bottom margin or explicit width/height in the same way.

4. What is the DOCTYPE declaration, and why is it needed?

`<!DOCTYPE html>` is a declaration at the top of an HTML document that tells the browser which version of HTML to use and triggers standards mode rendering. Without it, browsers may fall back to 'quirks mode', which emulates old, non-standard rendering behavior and can cause inconsistent layout.

5. What is semantic HTML, and why does it matter?

Semantic HTML means using elements according to their meaning rather than just their appearance — e.g. `<nav>`, `<article>`, `<header>`, `<footer>` instead of generic `<div>`s for everything. It matters because it improves accessibility (screen readers rely on semantic structure), SEO (search engines weigh semantic content more heavily), and code readability/maintainability.

Elements & Attributes

1. What is the difference between 'id' and 'class' attributes?

An 'id' uniquely identifies a single element on a page and should not be reused; it's often used for JavaScript hooks, fragment links, and label associations. A 'class' can be applied to multiple elements and is primarily used for grouping elements to share CSS styling or be selected together in scripts.

2. What are data attributes, and when would you use them?

Data attributes (e.g. `data-user-id="42"`) let you store custom data directly on HTML elements without resorting to non-standard attributes. They're accessed in JavaScript via the element's 'dataset' property (e.g. `element.dataset.userId`) and are commonly used to pass small bits of state between markup and scripts.

3. What is the difference between `<div>` and `<span>`?

`<div>` is a block-level, generic container used to group larger sections of content, typically for layout purposes. `<span>` is an inline, generic container used to wrap small pieces of text or inline elements, typically for styling or scripting a portion of text without breaking the flow.

4. What is the purpose of the 'alt' attribute on an `<img>` tag?

The 'alt' attribute provides alternative text describing an image. It's read aloud by screen readers for accessibility, displayed if the image fails to load, and used by search engines to understand image content. Decorative images that add no informational value should use an empty `alt=""` so screen readers skip them.

5. What are void (self-closing) elements? Give examples.

Void elements are elements that cannot have any content or closing tag because they represent a single, self-contained piece of markup. Examples include `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, and `<link>`.

Forms & Media

1. What is the difference between GET and POST methods in an HTML form?

GET appends form data to the URL as query parameters, making it visible, bookmarkable, and limited in length — suited to non-sensitive data retrieval like search forms. POST sends form data in the request body, keeping it hidden from the URL, with no practical length limit — suited to submitting sensitive or large data, such as login credentials or file uploads.

2. What is the purpose of the 'label' element, and how is it associated with a form control?

`<label>` provides a text caption for a form control and improves accessibility and usability — clicking the label focuses or activates the associated input. Association is done either by wrapping the input inside the label, or by setting the label's 'for' attribute to match the input's 'id'.

3. What are some common input types introduced in HTML5, and why are they useful?

HTML5 added input types like email, url, number, date, tel, range, color, and search. These provide built-in validation, more appropriate mobile keyboards, and native UI widgets (like date pickers), reducing the need for custom JavaScript validation and improving user experience.

4. What is the difference between the `<video>`/`<audio>` elements and using Flash or third-party plugins?

`<video>` and `<audio>` are native HTML5 elements that let browsers play media directly without any external plugin, using formats like MP4 and WebM/Ogg. This makes media playback more secure, better performing, accessible via standard APIs, and usable on devices (like mobile) that don't support plugins such as Flash, which is now deprecated entirely.

5. How does form validation work natively in HTML5?

HTML5 supports built-in validation through attributes like required, pattern, min/max, minlength/maxlength, and type-specific rules (e.g. a valid email format for `type="email"`). When a form is submitted, the browser checks these constraints automatically and shows default error messages, blocking submission until they're satisfied — without needing custom JavaScript.

Advanced & Best Practices

1. What is the difference between localStorage, sessionStorage, and cookies?

localStorage persists data in the browser with no expiration until explicitly cleared, and isn't sent with HTTP requests. sessionStorage behaves similarly but is cleared when the browser tab closes. Cookies are smaller (about 4KB), can have an expiration date, and are automatically sent with every HTTP request to the server, which makes them useful for authentication but adds overhead to each request.

2. What is the purpose of the <meta viewport> tag?

The viewport meta tag (e.g. <meta name="viewport" content="width=device-width, initial-scale=1">) controls how a page is scaled and sized on mobile devices. Without it, mobile browsers render the page at a desktop width and zoom out, resulting in tiny, hard-to-read text; it's essential for responsive design.

3. What is the difference between defer and async attributes on a <script> tag?

Both let a script download in parallel without blocking HTML parsing. 'async' scripts execute as soon as they finish downloading, potentially out of order relative to other scripts and before the DOM is fully parsed. 'defer' scripts execute only after the HTML document has been fully parsed, and multiple deferred scripts run in the order they appear in the document.

4. What are ARIA attributes, and when should you use them?

ARIA (Accessible Rich Internet Applications) attributes, like role, aria-label, and aria-hidden, add accessibility information for assistive technologies when native HTML semantics aren't sufficient — for example, on custom widgets built from <div>s. The general rule is to prefer native semantic HTML elements first, and only reach for ARIA when no native element provides the needed semantics.

5. What is the Shadow DOM, and what problem does it solve?

The Shadow DOM is a browser API that allows a component to have its own isolated, encapsulated DOM subtree with scoped CSS, separate from the main document. It solves the problem of style and markup leakage/collisions — commonly used in Web Components so a component's internal structure and styles don't clash with or get affected by the surrounding page.